MySQL for Data Analysts

COMPLETE STUDENT GUIDE

From Basics to Advanced — master SQL to query any database and extract powerful business insights.

- SQL Basics
- Filtering
- Joins
- Aggregations
- Subqueries
- Window Functions
- Real Projects

Made with GAMMA

Why MySQL for Data Analysts?

SQL is the **#1 most in-demand skill** for data analysts worldwide. Every company stores data in databases — you **MUST** know SQL to access it.

70%
Job Descriptions
Require SQL as skill #1

1M+
Rows in Seconds
Excel cannot do this

$0
Cost
MySQL is free and open-source

Daily
Usage
Analysts write SQL every single day

What You Can Do With MySQL

- Extract specific data from massive databases (millions of rows)
- Summarize and aggregate data by groups (region, category, month)
- Combine data from multiple tables using JOINs
- Find trends, patterns, and business insights
- Clean and transform data inside the database

SQL vs Other Tools

Excel	Max 1M rows, manual, visual
Python	Programmatic analysis, machine learning
MySQL	Databases, millions of rows, fastest for data extraction
Power BI	Visualization and dashboards (connects to MySQL)

Learn MySQL after Excel — it unlocks real enterprise-level data analysis.

Made with GAMMA

MySQL Setup — Install & Get Started

Tools to Install (All FREE)

1

MySQL Community Server

The database engine. Download from `dev.mysql.com/downloads/mysql` — choose your OS.

2

MySQL Workbench

The visual SQL editor (like Excel but for databases). Or install MySQL Installer which installs BOTH at once.

MySQL Workbench Interface

Left Panel	Shows databases and tables (like folders and files)
Center	SQL Editor — where you write queries
Bottom	Results panel — shows query output
Top Bar	Run button ⚡ or press CTRL + Enter to run query

Installation Steps

01

Download MySQL Installer (Windows) from `dev.mysql.com`

02

Run installer → choose **"Developer Default"** → Next → Execute

03

Set root password — remember this! (example: `Root@1234`)

04

Keep port as `3306` (default) → Finish

05

Open MySQL Workbench → click **"Local instance 3306"**

06

Enter your root password → Connect ✅

First Query to Test

```
SELECT "Hello Data Analyst!" AS message;
```

Run it → you should see: **Hello Data Analyst!** — MySQL is working! ✅

Made with GAMMA

Database Basics — Key Concepts

Before writing queries, understand these core concepts that form the foundation of every relational database.

DATABASE

A collection of related tables. Example: `retail_db` contains: sales, customers, products, employees

TABLE

Rows and columns — like an Excel sheet but far more powerful. Example: sales table has 10,000 rows and 11 columns

PRIMARY KEY

A unique identifier for each row (no duplicates, no nulls). Example: `order_id` is the primary key of the sales table

FOREIGN KEY

A column that links to the primary key of another table. Example: `customer_id` in sales → links to customers table

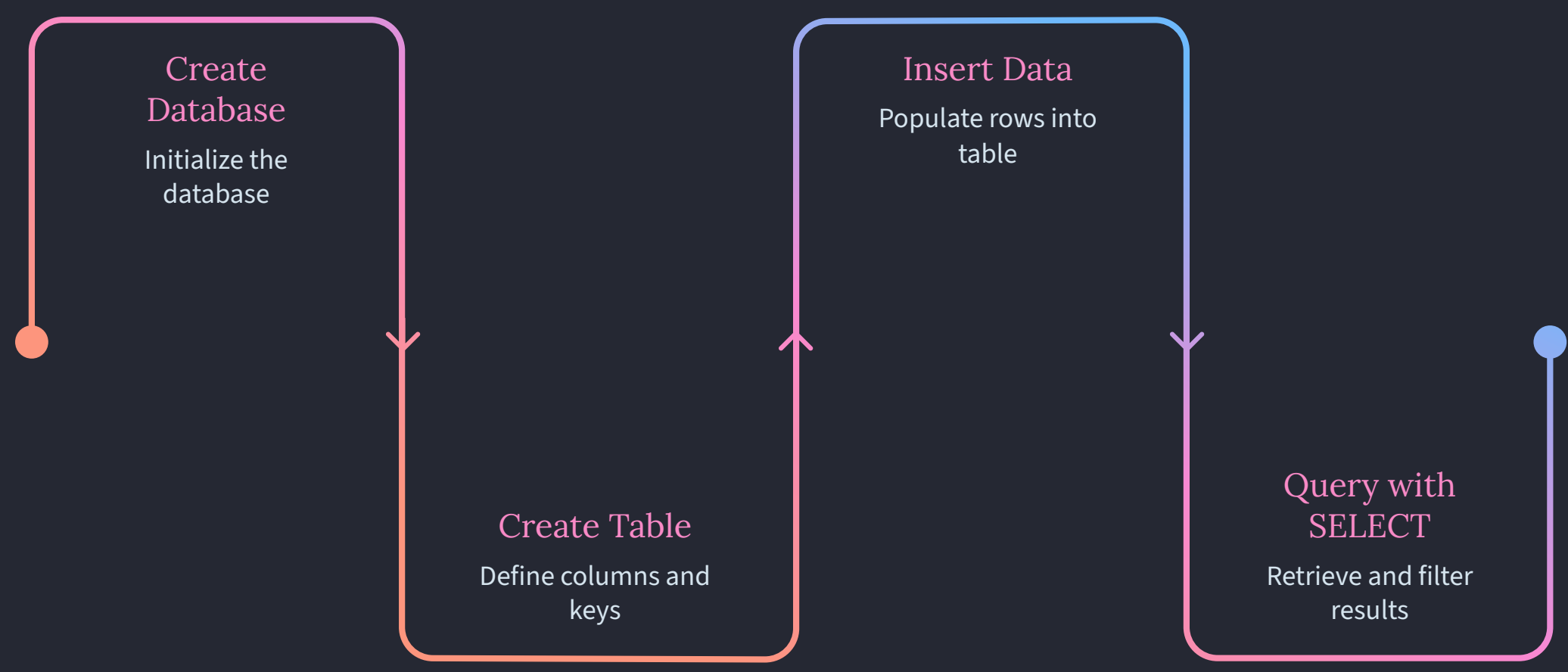
MySQL Data Types

INT	Whole numbers	1, 100, 9999
DECIMAL(10,2)	Decimal numbers	45000.50, 3.14
VARCHAR(100)	Text up to 100 chars	"Rahul Sharma"
DATE	Date values	2024-01-15
DATETIME	Date + time	2024-01-15 09:30:00
BOOLEAN	TRUE or FALSE	Stored as 1 or 0

Made with GAMMA

STAGE 1 — CREATE, INSERT, SELECT

BEGINNER



Follow these steps to build your first database and start querying data immediately.

Step 1–3: Create & Populate

```
CREATE DATABASE retail_db;
USE retail_db;

CREATE TABLE sales (
  order_id VARCHAR(20) PRIMARY KEY,
  order_date DATE,
  customer_name VARCHAR(100),
  region VARCHAR(50),
  category VARCHAR(50),
  product_name VARCHAR(100),
  units_sold INT,
  unit_price DECIMAL(10,2),
  total_sales DECIMAL(12,2),
  discount DECIMAL(5,2),
  profit DECIMAL(12,2)
);

INSERT INTO sales VALUES
('ORD001','2024-01-15','Rahul Sharma',
 'North','Electronics','Laptop Pro',
 2,45000,90000,5,18000),
('ORD002','2024-01-16','Priya Singh',
 'South','Clothing','Cotton Shirt',
 5,1200,6000,10,1200);
```

Step 4–6: Query Your Data

```
-- View all data
SELECT * FROM sales;

-- Select specific columns only
SELECT order_id, customer_name,
       total_sales, profit
FROM sales;

-- Import CSV for large datasets
LOAD DATA INFILE 'C:/retail_sales.csv'
INTO TABLE sales
FIELDS TERMINATED BY ','
LINES TERMINATED BY '\n'
IGNORE 1 ROWS;
```

Pro Tip: Use `SELECT *` only for exploration. In production queries, always specify the columns you need for better performance.

Made with GAMMA

STAGE 2–3 — WHERE, ORDER BY, LIMIT & DISTINCT

WHERE Clause — Filtering Data

WHERE = Filter rows based on conditions — the most used clause in SQL!

```
-- Filter by exact value
SELECT * FROM sales
WHERE region = 'North';

-- AND: both conditions true
SELECT * FROM sales
WHERE region = 'North'
  AND category = 'Electronics';

-- BETWEEN: range of values
SELECT * FROM sales
WHERE total_sales BETWEEN 5000 AND 50000;

-- IN: match any value in list
SELECT * FROM sales
WHERE region IN ('North','South','East');

-- LIKE: pattern matching
SELECT * FROM sales
WHERE customer_name LIKE 'R%';

-- IS NULL: find missing values
SELECT * FROM sales
WHERE profit IS NULL;
```

ORDER BY, LIMIT & DISTINCT

```
-- Sort descending (highest first)
SELECT * FROM sales
ORDER BY total_sales DESC;

-- Top 5 orders by sales
SELECT * FROM sales
ORDER BY total_sales DESC LIMIT 5;

-- Pagination: skip 10, show next 10
SELECT * FROM sales
LIMIT 10 OFFSET 10;

-- All unique regions
SELECT DISTINCT region FROM sales;

-- Count unique customers
SELECT COUNT(DISTINCT customer_name)
  AS unique_customers
FROM sales;
```

Comparison Operators

= Equal	!= Not equal
> Greater	< Less
>= Greater or equal	

Made with GAMMA

STAGE 4 — Aggregate Functions & GROUP BY

CORE ANALYTICS

Aggregate Functions calculate summary statistics on groups of data — the foundation of all business reporting.

COUNT()

Count rows in a group

SUM()

Add up all values

AVG()

Calculate the average

MIN()

Find the smallest value

MAX()

Find the largest value

Overall Business KPIs

```
SELECT
  COUNT(order_id) AS total_orders,
  SUM(total_sales) AS total_revenue,
  ROUND(AVG(total_sales),2) AS avg_order_value,
  MIN(total_sales) AS smallest_order,
  MAX(total_sales) AS largest_order,
  ROUND(SUM(profit)/SUM(total_sales)
      *100,2) AS profit_margin_pct
FROM sales;
```

Result: Orders: 10,000 | Revenue: ₹42L | Avg: ₹4,200 | Margin: 22%

GROUP BY + HAVING

```
-- Sales by region
SELECT region,
  COUNT(order_id) AS total_orders,
  SUM(total_sales) AS revenue,
  SUM(profit) AS profit
FROM sales
GROUP BY region
ORDER BY revenue DESC;

-- HAVING: filter groups
-- (like WHERE but for aggregated data)
SELECT region, SUM(total_sales) AS revenue
FROM sales
GROUP BY region
HAVING SUM(total_sales) > 1000000;
```

Key Rule: WHERE filters rows BEFORE grouping. HAVING filters groups AFTER grouping.

Made with GAMMA

STAGE 5–6 — Date Functions & JOINs

Key MySQL Date Functions

NOW()	Current date and time
CURDATE()	Current date only
YEAR(date)	Extract year number
MONTH(date)	Extract month number
MONTHNAME(date)	Month name (e.g. October)
QUARTER(date)	Quarter number (1–4)
DATE_FORMAT()	Custom format string
DATEDIFF()	Days between two dates

```
-- Monthly Sales Trend
SELECT
  YEAR(order_date) AS year,
  MONTHNAME(order_date) AS month_name,
  SUM(total_sales) AS monthly_revenue,
  ROUND(SUM(profit)/SUM(total_sales)
      *100,1) AS margin_pct
FROM sales
GROUP BY YEAR(order_date),
       MONTH(order_date),
       MONTHNAME(order_date)
ORDER BY year, MONTH(order_date);
```

JOINs — Combining Multiple Tables

INNER JOIN

Only matching rows from BOTH tables. Non-matching rows are excluded.

LEFT JOIN

ALL rows from LEFT table + matching from RIGHT. Shows NULL for missing matches.

RIGHT JOIN

ALL rows from RIGHT table + matching from LEFT.

FULL OUTER JOIN

All rows from BOTH tables. Use LEFT JOIN UNION RIGHT JOIN in MySQL.

```
-- 3-Table JOIN (most common in real work)
SELECT s.order_id, c.customer_name,
      p.product_name, p.category,
      s.total_sales, s.profit
FROM sales s
INNER JOIN customers c
  ON s.customer_id = c.customer_id
INNER JOIN products p
  ON s.product_id = p.product_id
ORDER BY s.total_sales DESC;
```

Made with GAMMA

STAGE 7–8 — Subqueries, CTEs & Window Functions

ADVANCED

Subqueries & CTEs

Subqueries nest one query inside another. CTEs (WITH clause) are a cleaner, more readable alternative.

```
-- Subquery: orders above average sales
SELECT order_id, customer_name, total_sales
FROM sales
WHERE total_sales > (
  SELECT AVG(total_sales) FROM sales
);

-- CTE: monthly revenue above average
WITH monthly_revenue AS (
  SELECT MONTHNAME(order_date) AS month,
    SUM(total_sales) AS revenue
  FROM sales
  GROUP BY MONTHNAME(order_date)
)
SELECT month, revenue
FROM monthly_revenue
WHERE revenue > (
  SELECT AVG(revenue) FROM monthly_revenue
)
ORDER BY revenue DESC;
```

Window Functions — Most Powerful Feature!

Perform calculations ACROSS rows without collapsing them into groups.

```
-- ROW_NUMBER: rank all orders
SELECT order_id, total_sales,
  ROW_NUMBER() OVER (
    ORDER BY total_sales DESC
  ) AS sales_rank
FROM sales;

-- Top 3 products per category
SELECT * FROM (
  SELECT category, product_name,
    SUM(total_sales) AS revenue,
    DENSE_RANK() OVER (
      PARTITION BY category
      ORDER BY SUM(total_sales) DESC
    ) AS rnk
  FROM sales
  GROUP BY category, product_name
) ranked WHERE rnk <= 3;

-- LAG: Month-over-Month growth %
SELECT MONTHNAME(order_date) AS month,
  SUM(total_sales) AS revenue,
  LAG(SUM(total_sales)) OVER (
    ORDER BY MONTH(order_date)
  ) AS prev_month,
  ROUND((SUM(total_sales) - LAG(SUM(total_sales))
    OVER (ORDER BY MONTH(order_date)))
    / LAG(SUM(total_sales))
    OVER (ORDER BY MONTH(order_date))
    * 100, 1) AS mom_growth_pct
FROM sales
GROUP BY MONTHNAME(order_date),
       MONTH(order_date);
```

ROW_NUMBER()	RANK()	DENSE_RANK()
Sequential unique numbers — no ties	Skips numbers after ties: 1,2,2,4	No gaps after ties: 1,2,2,3

SUM() OVER	LAG() / LEAD()
Running cumulative total	Compare with previous/next row

Made with GAMMA

STAGE 9–10 — CASE Statements & String Functions

CASE Statement — IF-ELSE Logic in SQL

Creates new categories from existing data — essential for segmentation and pivots.

```
-- Order segmentation
SELECT order_id, total_sales,
  CASE
    WHEN total_sales >= 50000 THEN 'High Value'
    WHEN total_sales >= 10000 THEN 'Medium Value'
    WHEN total_sales >= 1000  THEN 'Low Value'
    ELSE 'Micro Order'
  END AS order_segment
FROM sales;

-- PIVOT: Category sales by region
SELECT category,
  SUM(CASE WHEN region='North'
    THEN total_sales ELSE 0 END) AS North,
  SUM(CASE WHEN region='South'
    THEN total_sales ELSE 0 END) AS South,
  SUM(CASE WHEN region='East'
    THEN total_sales ELSE 0 END) AS East,
  SUM(CASE WHEN region='West'
    THEN total_sales ELSE 0 END) AS West,
  SUM(total_sales) AS Total
FROM sales GROUP BY category;
```

String Functions — Clean & Manipulate Text

UPPER() / LOWER()	Change text case
LENGTH()	Count characters
TRIM()	Remove leading/trailing spaces
SUBSTRING()	Extract part of text
CONCAT()	Join strings together
REPLACE()	Find and replace text
REGEXP	Regular expression matching

```
-- CONCAT example
SELECT CONCAT(customer_name,' - ',region)
  AS customer_region FROM sales;
-- → 'Rahul Sharma - North'

-- SUBSTRING example
SELECT SUBSTRING(order_id, 4, 3) FROM sales;
-- 'ORD001' → extracts '001'

-- LIKE wildcards
-- % = any number of characters
-- _ = exactly one character
SELECT * FROM sales
WHERE product_name LIKE 'Laptop%';

-- REGEXP: names starting with A or R
SELECT * FROM sales
WHERE customer_name REGEXP '^[AR]';
```

Made with GAMMA